

Java 7

Les « nouveautés »

Katy Saintin
Novembre 2015



- Opérateur diamant
- Instruction Switch case avec des String
- Fermeture automatique des ressources
- Utilisation de « _ » dans les entiers littéraux
- Amélioration sur la gestion des exceptions
- API NIO d'accès au système de fichiers
- Framework fork / join
- Rethrow
- Support des langages dynamiques

Ressource :

<http://adiguba.developpez.com/tutoriels/java/7/>

<http://radar.oreilly.com/2011/09/java7-features.html>

Avant :

```
Map<String, List<MyObject>> trades = new TreeMap<String, List<MyObject>> ();
```

Java 7 :

```
Map<String, List<MyObject>> trades = new TreeMap<> ();
```



Avant :

```
String state = device.state();  
if(status.equalsIgnoreCase("OPEN")) {  
    label.setBackground(COLOR.GREEN);  
} else if(status.equalsIgnoreCase("CLOSE")) {  
    label.setBackground(COLOR.WHITE);  
} else if(status.equalsIgnoreCase("STANDBY")) {  
    label.setBackground(COLOR.YELLOW);  
}
```

Java 7 :

```
String state = device.state();  
switch(status) {  
    case "OPEN" : label.setBackground(COLOR.GREEN);break;  
    case "CLOSE" : label.setBackground(COLOR.WHITE);break;  
    case "STANDBY" : label.setBackground(COLOR.YELLOW);break;  
}
```

Avant :

```
try {
    BufferedReader input = null;
    try {
        input = new BufferedReader(new FileReader(fichier));
        String ligne = null;
        while ((ligne = input.readLine()) != null) {
            contenu.append(ligne);
            contenu.append("\n");
        }
    } finally {
        if (input != null) {

            try {
                input.close();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
} catch (IOException ex) {
    ex.printStackTrace();
}
```

Java 7 :

```
try {
    BufferedReader input = new BufferedReader(new FileReader(fichier);
    try (input) {
        String ligne=null;
        while ((ligne = bufferedReader.readLine()) != null) {
            System.out.println(ligne);
        }
    }
} catch (IOException ioe) {
    ioe.printStackTrace();
}
```

Pour afficher des groupe de chiffres et séparer les milliers, les millions, les milliards afin d'améliorer la lisibilité du code.

Exemple (code Java 7) :

```
int maValeur = 123_1456_789;  
maValeur = 4_3;  
maValeur = 4___3;  
maValeur = 0x4_3;  
maValeur = 0_43;  
maValeur = 04_3;  
maValeur = 0b1001110_10001011_01001101_01000101;  
long creditCardNumber = 1234_5678_9012_3456L;  
long numeroSecuriteSociale = 1_75_02_31_235_897L;  
long octetsDebutFichierClass = 0xCAFE_BABE;  
long maxLong = 0x7fff_ffff_ffff_ffffL;  
float pi = 3.141_593f;
```

Avant :

```
public void oldMultiCatch() {  
    try{  
        methodThatThrowsThreeExceptions();  
    } catch(ExceptionOne e) {  
        // log and deal with ExceptionOne  
    } catch(ExceptionTwo e) {  
        // log and deal with ExceptionTwo  
    } catch(ExceptionThree e) {  
        // log and deal with ExceptionThree  
    }  
}
```

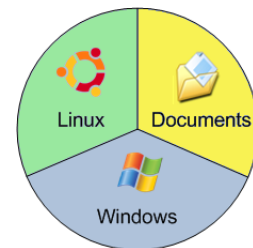
Java 7 :

```
public void newMultiCatch() {  
    try{  
        methodThatThrowsThreeExceptions();  
    } catch(ExceptionOne | ExceptionTwo | ExceptionThree e) {  
        // log and deal with all Exceptions  
    }  
}
```

API NIO 2.0 plus moderne et plus complète pour l'accès au système de fichiers. Son but est en partie de remplacer la classe File de la très ancienne API IO.

- étend les fonctionnalités relatives aux entrées/sorties.
- simplifie la manipulation de fichiers et répertoires d'un système de fichiers
- facilite les lectures/écritures asynchrones
- supporte les liens physiques et symboliques
- gère les notifications en cas de changement d'un répertoire (API WatchService)
- parcourt un répertoire avec la possibilité de filtrer les fichiers obtenus
- lecture/écriture en utilisant un pool de threads
- propose des fonctionnalités de base comme la copie ou le déplacement de fichiers
- permet l'utilisation de fabriques pour étendre l'API (implémentation permettant de gérer les fichiers zip fournie en standard).

NIO2 est probablement l'API de Java 7 qui sera la plus utilisée par les développeurs tant elle facilite la mise en œuvre de fonctionnalités courantes d'entrées/sorties sur un système de fichiers.




```

public void watchDirectoryModifications() {
    try {
        WatchService watchService = FileSystems.getDefault().newWatchService();
        final Path dir = Paths.get("c:/java/test");

        //Register modification
        WatchKey key = dir.register(watchService,
                                    StandardWatchEventKinds.ENTRY_CREATE,
                                    StandardWatchEventKinds.ENTRY_DELETE,
                                    StandardWatchEventKinds.ENTRY_MODIFY);

        boolean running = true;
        while (running) {
            try {
                key = watchService.poll(1000, TimeUnit.MILLISECONDS);
                if (key != null) {
                    for (final WatchEvent<?> event : key.pollEvents()) {
                        final Path name = (Path) event.context();
                        System.out.format(event.kind() + " " + "%s\n", name);
                    }
                    key.reset();
                }
            } catch (final InterruptedException e) {
                e.printStackTrace();
                running = false;
            }
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}

```

Objectifs :

Ce Framework permet d'exécuter une tâche volumineuse en plusieurs tâches plus petites. (package `java.util.concurrent`)

- Classe **ForkJoinTask**= tâche unique. Peut générer d'autres tâches élémentaires. Est envoyée à une réserve de threads.
- Classe **ForkJoinPool** = réserve de threads. Conçue pour gérer un très grand nombre de tâches élémentaires.

Différence (algorithmique) entre Fork/Join et Executor :

Contrairement à l'Executor, quand une tâche attend la finalisation de ses sous-tâches créées par un join, le thread va chercher d'autres tâches encore non exécutées pour les démarrer. Du coup, les performances sont nettement meilleures.

Attention, cela peut être gourmand au niveau processeur.



TAKIPI

Exemple d'implémentation :

En Java SE 8, utilisé par `java.util.Arrays` pour `parallelSort()`

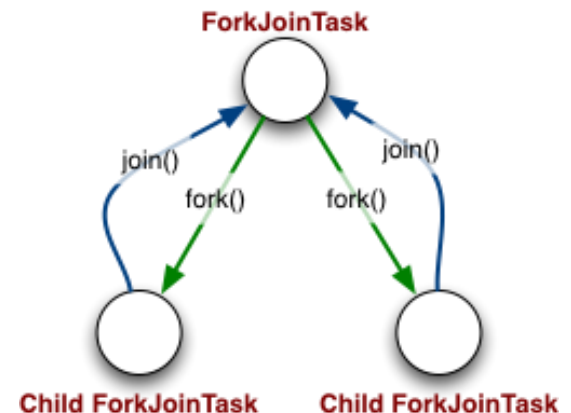
- **ForkJoinTask**

- ✓ `join()` : retourne le résultat de l'exécution de cette tâche.
- ✓ `fork()` : lance une autre tâche dans la même réserve qu'elle.

- **ForkJoinPool**

- ✓ `execute(ForkJoinTask<?> task)` : déclenche l'exécution asynchrone de la tâche passée en paramètre ;
- ✓ `invoke(ForkJoinTask<?> task)` : déclenche l'exécution synchrone de la tâche passée en paramètre, cet appel ne rend la main que lorsque la tâche est exécutée
- ✓ `submit(ForkJoinTask<?> task)` : retourne immédiatement un objet de type `Future<V>`.

Même sémantique de lancement de threads.



?